

POST MORTEM

Incidente: Deadlocks y Timeouts en Pico de Quincena

SmartBancs App — Reto Técnico TCS | Erick Ballas

Fecha del incidente: 15 de septiembre de 2026

RESUMEN

El día 15 de septiembre de 2026, durante el pico de quincena (14:30 – 15:17 UTC-5), la plataforma SmartBancs App experimentó degradación severa del servicio transaccional. Aproximadamente 2,300 transacciones sufrieron latencia > 5 segundos y 180 transacciones fallaron. La causa raíz fue una cadena de deadlocks en PostgreSQL provocada por la ausencia de un índice en la tabla transactions durante un pico de 8x el volumen normal.

1. Métricas del Incidente

Métrica	Valor Normal	Valor Durante Incidente
Latencia p99 (transacciones)	< 500ms	> 5,200ms
Tasa de error	< 0.1%	7.8% (180/2,300 tx)
TPS procesados	~1,200 TPS	~340 TPS (caída del 72%)
Cola RabbitMQ (tx.completed)	< 100 mensajes	14,500 mensajes acumulados
Conexiones activas PostgreSQL	~80	487 (límite de pool)
Duración del incidente	N/A	47 minutos (14:30 – 15:17)
Usuarios afectados	N/A	~1,800 usuarios activos

2. Timeline del Incidente

Hora (UTC-5)	Evento	Responsable
14:30	Pico de quincena inicia. TPS sube de 1,200 a 9,800 en 4 minutos.	Sistema
14:33	Prometheus alerta: p99 > 1.5s (umbral de warning). Primer deadlock detectado en pg_stat_activity.	Alerta automática

Hora (UTC-5)	Evento	Responsable
14:38	Escalada al equipo de ingeniería. Pool de conexiones PostgreSQL saturado (487/500).	On-call engineer
14:42	Diagnóstico: pg_blocking_pids revela cadena de 23 transacciones bloqueadas. Full table scan identificado en EXPLAIN ANALYZE.	Erick Ballas
14:45	Acción inmediata: pg_terminate_backend() en los 3 PIDs bloqueantes principales. Latencia baja a 2.1s.	Erick Ballas
14:48	Circuit breaker activado: lecturas de saldo redirigidas a Redis. Sync-worker pausado para reducir contención.	Erick Ballas
14:55	Creación del índice (CONCURRENTLY) en producción: CREATE INDEX CONCURRENTLY idx_tx_status ON transactions(status, created_at).	Erick Ballas
15:05	Índice completado. p99 baja a 480ms. Tasa de error cae a 0.3%.	Sistema
15:10	Cola RabbitMQ procesándose. Transacciones encoladas completándose.	AI Worker
15:17	Sistema completamente estable. p99 = 280ms. Tasa de error = 0.0%. Incidente cerrado.	Erick Ballas

3. Análisis de Causa Raíz

3.1 Causa Raíz Principal

CAUSA RAÍZ

Ausencia de un índice parcial en la tabla transactions para la columna status con created_at. Durante el pico, las validaciones de historial ejecutaban full table scans en una tabla con > 500,000 registros, manteniendo locks de tabla durante > 2,000ms. La combinación de locks prolongados + 487 conexiones simultáneas generaron una cadena de deadlocks.

3.2 Factores Contribuyentes

- Pool de conexiones sin límite por servicio: Todas las conexiones disponibles (500) fueron consumidas, imposibilitando que nuevas transacciones obtuvieran conexión.
- Ausencia de READ REPLICA: Queries de lectura de saldo competían con escrituras en el mismo servidor PostgreSQL.
- Timeout de query muy largo: maxQueryExecutionTime de 500ms no generaba alerta temprana suficiente para el pico.

- Sin load testing previo al pico de quincena: El comportamiento bajo 10K TPS no fue simulado antes del evento.
- Ausencia de runbook de pico: No existía procedimiento documentado para escenarios de quincena.

4. Lecciones Aprendidas

Este análisis se enfoca en factores sistémicos y de proceso, no en responsabilidades individuales. El objetivo es mejorar el sistema para que ninguna persona necesite heroicamente intervenir bajo presión.

¿Qué funcionó bien?

- El ordenamiento canónico de UUIDs evitó que el escenario fuera peor (deadlocks cruzados eliminados a nivel de código).
- Redis absorbió las lecturas de saldo durante la crisis — sin él, la degradación habría sido total.
- El AI Worker nunca afectó las transacciones (diseño async-first demostrado en producción).
- Los logs estructurados con TraceID permitieron reconstruir el timeline en < 8 minutos.

¿Qué podría haber ido mejor?

- El índice debió crearse antes del lanzamiento con pruebas de carga previas.
- Las alertas de warning (1.5s) debieron escalar automáticamente, no manualmente.
- El pool de conexiones debió tener límites por servicio desde el inicio.

5. Plan de Acciones Correctivas

Acción	Tipo	Prioridad	Responsable	Fecha Límite
Crear índice idx_tx_status en transactions (status, created_at)	Código/DB	P0 - CRÍTICA	Erick Ballas	Completado
Configurar PgBouncer: max_client_conn = 200, pool_size = 20 por servicio	Infraestructura	P0 - CRÍTICA	DevOps	Oct 2026
Implementar load test automatizado (k6): 10K TPS x 5 min antes de cada quincena	Proceso	P1 - ALTA	QA Team	Oct 2026
Configurar escalado automático de alert p99 > 800ms → PagerDuty	Observabilidad	P1 - ALTA	SRE Team	Oct 2026

Acción	Tipo	Prioridad	Responsable	Fecha Límite
Deploy de Read Replica PostgreSQL para lecturas de historial	Infraestructura	P2 - MEDIA	Erick Ballas	Nov 2026
Documentar Runbook: procedimiento de pico de quincena	Proceso	P2 - MEDIA	Erick Ballas	Oct 2026
Revisar y ajustar todos los índices con EXPLAIN ANALYZE en producción	DB	P2 - MEDIA	DBA	Nov 2026

6. Métricas de Éxito Post-Incidente

Las siguientes métricas definen que el plan correctivo fue exitoso:

- p99 de latencia transaccional < 500ms durante el próximo pico de quincena.
- Tasa de error < 0.1% durante 10K TPS sostenidos por 10 minutos.
- Zero deadlocks detectados en pg_stat_activity durante el pico.
- Cola RabbitMQ < 500 mensajes durante el pico (AI Worker procesando sin atraso).